Computer Programming (2)

Lecture No.1: **Arrays Review**

Lecturer: Dr. Khaled Mohammed bin Abdl
Department of Information Technology,
Faculty of Applied Science,
Seyion University

Semester 2, 2023

Content

- One-dimensional arrays
- Array elements
- Approaches of initialization arrays
- Out-of-Bound error
- Array of characters
- Passing arrays to functions
- Searching arrays
- Multidimensional arrays

Array

- Data structure containing related data items of the same type
  - Hold in consecutive group of memory locations
- "static" entity remain the same size throughout program execution
- Syntax: `dataType arrayName[arraySize];`
  - `int c[5];`

Array size (Number of elements)

Name of array (any valid identifier)

Type of array elements

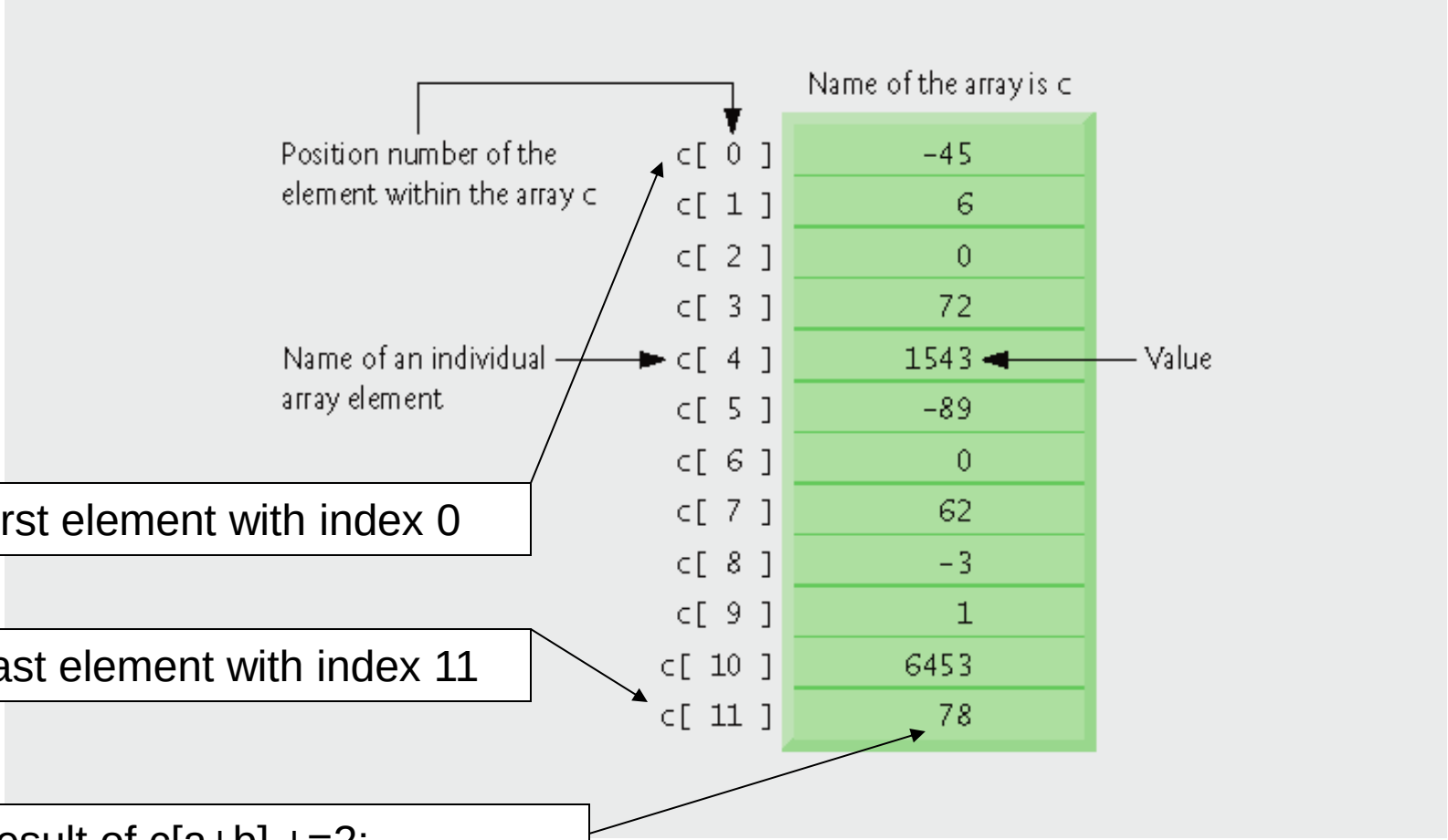
c[0]	?
c[1]	?
c[2]	?
c[3]	?
c[4]	?

Array Elements

- Data items in the array
- Each element has a position in the array called index or subscript
- First element has index 0 and last element has index arraySize-1
- To access any element, giving array name followed by index in [ ]
  - `c[0], c[1],` …etc
  - [ ] is an operator has same precedence and associativity of ( )
- Position number must be
  - Positive integer **OR** any expression that results in a positive integer

```
int a=5, b=6;
c[a+b] +=2; // Adds 2 to array element c[ 11 ]
```

Array Elements (cont.)



First element with index 0

Last element with index 11

Result of c[a+b] +=2;

will store 80 in c[11]

Array Initialization – Approach (1)

- Using a loop to initialize array's elements

```
// Fig. 7.3: fig07_03.cpp
// Initializing an array.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   int n[ 10 ]; // n is an array of 10 integers

   // initialize elements of array n to 0
   for ( int i = 0; i < 10; i++ )
      n[ i ] = 0; // set element at location i to 0

   cout << "Element" << setw( 13 ) << "Value" << endl;
```

Declare **n** as an array of **int**s with 10 elements

Each **int** initialized is to **0**

```
19
20    // output each array element's value
21    for ( int j = 0; j < 10; j++ )
22       cout << setw( 7 ) << j << setw( 13 ) << n[ j ] << endl;
23
24    return 0; // indicates successful termination
25 } // end main
```

```
Element        Value
      0            0
      1            0
      2            0
      3            0
      4            0
      5            0
      6            0
      7            0
      8            0
      9            0
```

n[j] returns **int** associated with index **j** in array **n**

Each **int** has been initialized to **0**

```cpp
// Fig. 7.4: fig07_04.cpp
// Initializing an array in a declaration.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   // use initializer list to initialize array n
   int n[ 10 ] = { 32, 27, 64, 18, 95, 14, 90, 70, 60, 37 };

   cout << "Element" << setw( 13 ) << "Value" << endl;

   // output each array element's value
   for ( int i = 0; i < 10; i++ )
      cout << setw( 7 ) << i << setw( 13 ) << n[ i ] << endl;

   return 0; // indicates successful termination
} // end main
```

```
Element         Value
      0            32
      1            27
      2            64
      3            18
      4            95
      5            14
      6            90
      7            70
      8            60
      9            37
```

Array Initialization – Approach(2)

- In declaration by using initializer list
  - Initializer list
    - Items enclosed in braces {}
    - Items in list separated by commas
    - Items themselves called initializers

```
int c[5] = { 10, 20, 30, 40, 50 };
```

c[0]	10
c[1]	20
c[2]	30
c[3]	40
c[4]	50

- If initializers are **more** than array elements (size)
  - Produce a compilation error

```
int c[5] = { 10, 20, 30, 40, 50, 60, 70 };
```

Array Initialization – Approach(2) (cont.)

- If initializers are **fewer** than array elements (size)
  - Remaining elements are initialized to zero

```
int c[5] = { 11 };
```

c[0]	11
c[1]	0
c[2]	0
c[3]	0
c[4]	0

- To initialize all array elements to 0
  - Explicitly initializes first element to zero
  - Implicitly initializes remaining four elements to zero

```
int x[5] = { 0 };
```

x[0]	0
x[1]	0
x[2]	0
x[3]	0
x[4]	0

  - Static array is initialized to zero by compiler
    - No need to initialized explicitly

Array Initialization – Approach(2) (cont.)

- Omitting array size in declaration, compiler determines array size based on number of items in initializer list

```
int c[] = { 10, 20, 30, 40, 50 };
```

  - How many elements in array c ? **OR** Array size?
  - Index values?
  - Initialized values?

Exercises :

Write C++ statements to accomplish each of the following:

a) Display the value of the seventh element of array **`f`**.

b) Input a value into element 4 of single-subscripted array **`b`**.

Array size As Constant variable


```
// Fig. 7.5: fig07_05.cpp
// Set array s to the even integers from 2 to 20.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   // constant variable can be used to specify array size
   const int arraySize = 10;

   int s[ arraySize ]; // array s has 10 elements

   for ( int i = 0; i < arraySize; i++ ) // set the values
      s[ i ] = 2 + 2 * i;

   cout << "Element" << setw( 13 ) << "Value" << endl;

   // output contents of array s in tabular format
   for ( int j = 0; j < arraySize; j++ )
      cout << setw( 7 ) << j << setw( 13 ) << s[ j ] << endl;

   return 0; // indicates successful termination
} // end main
```

Declare constant variable **arraySize** using the **const** keyword

Declare array that contains 10 **int**s

Use array index to assign element's value

Element	Value
0	2
1	4
2	6
3	8
4	10
5	12
6	14
7	16
8	18
9	20

Example: Summation of the array elements


```
// Fig. 7.8: fig07_08.cpp
// Compute the sum of the elements of the array.
#include <iostream>
using std::cout;
using std::endl;

int main()
{
   const int arraySize = 10; // constant variable indicating size of array
   int a[ arraySize ] = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };
   int total = 0;

   // sum contents of array a
   for ( int i = 0; i < arraySize; i++ )
      total += a[ i ];

   cout << "Total of array elements: " << total << endl;

   return 0; // indicates successful termination
} // end main
```

Declare array with initializer list

Sum all array values

```
Total of array elements: 849
```

Out-of-Bound Error

- C++ has no array bounds checking
  - Does not prevent computer from referring to an element that does not exist

- Logic error
  - Referring to an element outside the array bounds

```
int main()
{
   const int arraySize = 10; // constant variable indicating size of array
   int a[ arraySize ] = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };
   int total = 0;

   // sum contents of array a
   for ( int i = 0; i < arraySize; i++ )
      total += a[ i ];   i <= arraySize

   cout << "Total of array elements: " << total << endl;
```

Array of Characters (String)

- String is actually array of characters
- Using character arrays to store and manipulate strings
- Can be initialized using a string literal

```
char str[] = "first";
```

  - Array size is length of string **plus** a special string-termination character called the null character ('\0')

- Also can be initialized with individual character constants (with single quote) in an initializer list

```
char str[] = {'f','i','r','s','t','\0'};
```

  - Not providing null character produce logic error

Array of Characters (String) (cont.)

- It is possible to input a string directly into a character array from the keyboard using `cin`

```
char string1[20];
cin >> str;
```

  - `cin >>` may read more characters than the array can store which leads to lose data (logic error)

- Characters in array (string) are output with `cout<<` until null-terminated character is encountered

```cpp
// Fig. 7.12: fig07_12.cpp
// Treating character arrays as strings.
#include <iostream>
using std::cout;
using std::cin;
using std::endl;

int main()
{
   char string1[ 20 ]; // reserves 20 characters
   char string2[] = "string literal"; // reserves 15 characters

   // read string from user into array string1
   cout << "Enter the string \"hello there\": ";
   cin >> string1; // reads "hello" [space terminates input]

   // output strings
   cout << "string1 is: " << string1 << "\nstring2 is: " << string2;

   cout << "\nstring1 with spaces between characters is:\n";

```

Store **"string literal"** as an array of characters

Initializing an array of characters using **cin**

Output array using **cin**

```
   // output characters until null character is reached
   for ( int i = 0; string1[ i ] != '\0'; i++ )
      cout << string1[ i ] << ' ';

   cin >> string1; // reads "there"
   cout << "\nstring1 is: " << string1 << endl;

   return 0; // indicates successful termination
} // end main
```

Loop until the terminating null character is reached

Accessing specific characters in the array

```
Enter the string "hello there": hello there
string1 is: hello
string2 is: string literal
string1 with spaces between characters is:
h e l l o
string1 is: there
```

Find the error(s) in each of the following statement:

a) Assume that: `char f[ 5 ];`

```
cin >> f; // User types hello
```

Passing Arrays to Functions

- Passing array as argument to function
  - by using array name without brackets [ ]

```
int c[5];        modifyArray(c, 5);
```

  - To process elements within function
    - Array size passed as argument
    - OR Place array size as global variable

- By default, arrays are passed by Reference
  - Called function can modify elements' values in original array
  - Function call actually passes starting address of array
    - So function knows where array is located in memory

Passing Arrays to Functions (cont.)

- Function parameter list must specify array parameter by [ ]

```
void modifyArray(int c[], int arraySize){}
```

  - Equivalent prototype

```
void modifyArray(int [], int);
```

- Array parameter may include the size of the array
  - Compiler will ignore it, though
    - Compiler only cares about the address of the first element

```
void modifyArray(int c[11], int arraySize){}
```

```cpp
// Fig. 7.14: fig07_14.cpp
// Passing arrays and individual array elements to functions.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

void modifyArray( int [], int ); // appears strange
void modifyElement( int );

int main()
{
   const int arraySize = 5; // size of array a
   int a[ arraySize ] = { 0, 1, 2, 3, 4 }; // initialize array a

   cout << "Effects of passing entire array by reference:"
      << "\n\nThe values of the original array are:\n";

   // output original array elements
   for ( int i = 0; i < arraySize; i++ )
      cout << setw( 3 ) << a[ i ];

   cout << endl;

   // pass array a to modifyArray by reference
   modifyArray( a, arraySize );
   cout << "The values of the modified array are:\n";
```

Function takes an array as argument

Declare **5-int** array **array** with initializer list

Pass entire array to function `modifyArray`

```
   // output modified array elements
   for ( int j = 0; j < arraySize; j++ )
      cout << setw( 3 ) << a[ j ];

   cout << "\n\n\nEffects of passing array element by value:"
      << "\n\na[3] before modifyElement: " << a[ 3 ] << endl;

   modifyElement( a[ 3 ] ); // pass array element a[ 3 ] by value
   cout << "a[3] after modifyElement: " << a[ 3 ] << endl;

   return 0; // indicates successful termination
} // end main

// in function modifyArray, "b" points to the original arra
void modifyArray( int b[], int sizeOfArray )
{
   // multiply each array element by 2
   for ( int k = 0; k < sizeOfArray; k++ )
      b[ k ] *= 2;
} // end function modifyArray
```

Pass array element **a[3]** to function **modifyElement**

Function **modifyArray** manipulates the array directly

```
// in function modifyElement, "e" is a local copy of
// array element a[ 3 ] passed from main
void modifyElement( int e )
{
   // multiply parameter by 2
   cout << "Value of element in modifyElement: " << ( e *= 2 ) << endl;
} // end function modifyElement
```

Function **`modifyElement`** manipulates array element's copy

```
Effects of passing entire array by reference:

The values of the original array are:
  0  1  2  3  4
The values of the modified array are:
  0  2  4  6  8


Effects of passing array element by value:

a[3] before modifyElement: 6
Value of element in modifyElement: 12
a[3] after modifyElement: 6
```

Passing Arrays to Functions (cont.)

- Since passing array to function by reference
  - Called function can modify elements' values
  - To prevent modification of array in called function
    - Precede array parameter by `const` qualifier

```
Void modifyArray(const int [], int);
```

```cpp
// Fig. 7.15: fig07_15.cpp
// Demonstrating the const type qualifier.
#include <iostream>
using std::cout;
using std::endl;

void tryToModifyArray( const int [] ); // function prototype

int main()
{
   int a[] = { 10, 20, 30 };

   tryToModifyArray( a );
   cout << a[ 0 ] << ' ' << a[ 1 ] << ' ' << a[ 2 ] << '\n';

   return 0; // indicates successful termination
} // end main

// In function tryToModifyArray, "b" cannot be used
// to modify the original array "a" in main.
void tryToModifyArray( const int b[] )
{
   b[ 0 ] /= 2; // error
   b[ 1 ] /= 2; // error
   b[ 2 ] /= 2; // error
} // end function tryToModifyArray
```

Using **const** to prevent the function from modifying the array

Array **a** will be **const** when in the body of the function

Array cannot be modified; it is **const** within the body function

Searching Arrays

- Process of finding a particular element n(key value OR search key) in the array
- Linear search is one of searching techniques
  - Compares each element of array with search key
  - Work well with small arrays OR unsorted arrays
  - Inefficient for large arrays
- Binary search is another searching technique
  - Can be used with sorted arrays

```cpp
// Fig. 7.19: fig07_19.cpp
// Linear search of an array.
#include <iostream>
using std::cout;
using std::cin;
using std::endl;

int linearSearch( const int [], int, int ); // prototype

int main()
{
   const int arraySize = 100; // size of array a
   int a[ arraySize ]; // create array a
   int searchKey; // value to locate in array a

   for ( int i = 0; i < arraySize; i++ )
      a[ i ] = 2 * i; // create some data

   cout << "Enter integer search key: ";
   cin >> searchKey;

   // attempt to locate searchKey in array a
   int element = linearSearch( a, searchKey, arraySize );

```

Function takes an array, a key value, and the size of the array as arguments

Function returns location of key value, **-1** if not found

```
   // display results
   if ( element != -1 )
      cout << "Found value in element " << element << endl;
   else
      cout << "Value not found" << endl;

   return 0; // indicates successful termination
} // end main

// compare key to every element of array until location is
// found or until end of array is reached; return subscript of
// element if key or -1 if key not found
int linearSearch( const int array[], int key, int sizeOfArray )
{
   for ( int j = 0; j < sizeOfArray; j++ )
      if ( array[ j ] == key ) // if found,
         return j; // return location of key

   return -1; // key not found
} // end function linearSearch
```

Search through entire array

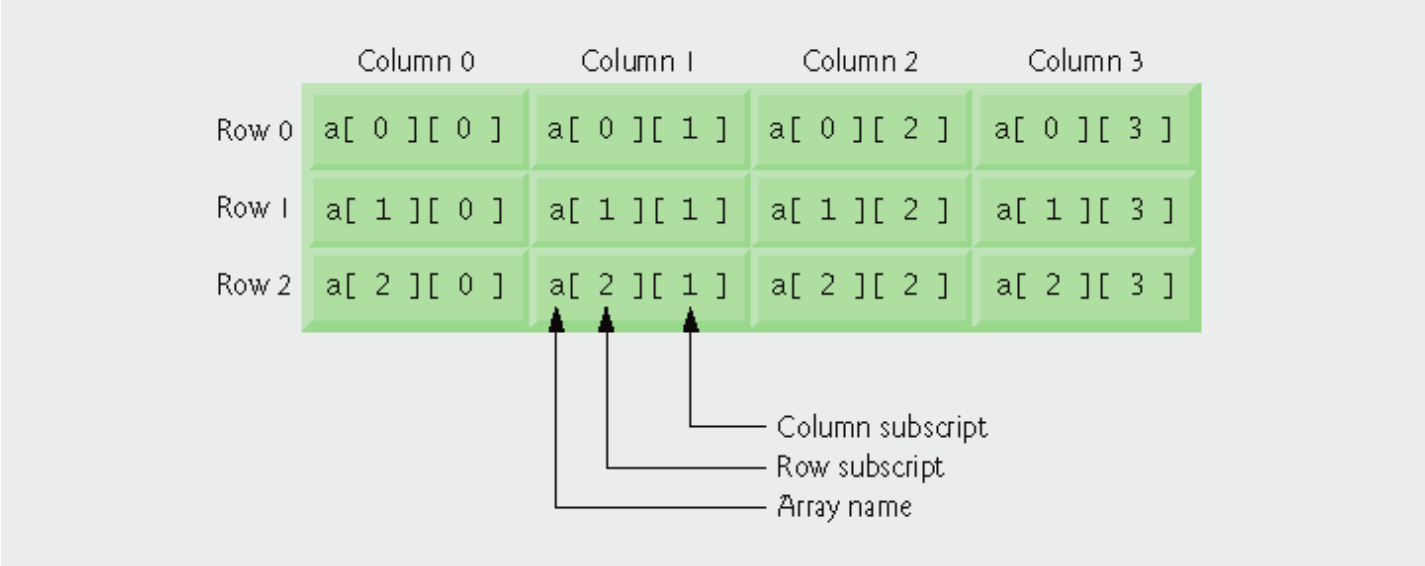
Return location if current value equals key value

```
Enter integer search key: 36
Found value in element 18
```

```
Enter integer search key: 37
Value not found
```

Multidimensional Array

- Multidimensional arrays with **two** dimensions
  - Called two dimensional or 2-D arrays
  - Represent tables of values with rows and columns
  - Elements referenced with two subscripts ([x][y])
  - Referencing a 2-D array element as c[x][y], incorrectly as c[x, y]



Initialization of Multidimensional Array

- Can be initialized in declaration much like one-dimension array
  - `int b[ 2 ][ 2 ] = { { 1, 2 }, { 3, 4 } };`
    - 1 and 2 initialize `b[ 0 ][ 0 ]` and `b[ 0 ][ 1 ]`
    - 3 and 4 initialize `b[ 1 ][ 0 ]` and `b[ 1 ][ 1 ]`
  - `int b[ 2 ][ 2 ] = { { 1 }, { 3, 4 } };`
    - Row 0 contains values 1 and 0 (implicitly initialized to zero)
    - Row 1 contains values 3 and 4
  - `int b[ 2 ][ 2 ] = { 1, 2, 3, 4 };`
    - 1 and 2 initialize `b[ 0 ][ 0 ]` and `b[ 0 ][ 1 ]`
    - 3 and 4 initialize `b[ 1 ][ 0 ]` and `b[ 1 ][ 1 ]`

Exercises :

a) Write a declaration for 2-D array called **`t`**.

b) How many rows does **`t`** have?

c) How many columns does **`t`** have?

d) How many elements does **`t`** have?

e) Write the names of all the elements in the second row of **`t`**.

f) Write the names of all the elements in the third column of **`t`**.

g) Write a single statement that sets the element of **`t`** in row 1 and column 2 to zero.

Multidimensional Array Parameters

- Size of first dimension is not required
  - As with a one-dimensional array
- Size of subsequent dimension is required
  - Compiler must know how many elements to skip to move to the second row

```
void printArray( const int a[][ 3 ] );
```

```cpp
// Fig. 7.22: fig07_22.cpp
// Initializing multidimensional arrays.
#include <iostream>
using std::cout;
using std::endl;

void printArray( const int [][ 3 ] ); // prototype

int main()
{
   int array1[ 2 ][ 3 ] = { { 1, 2, 3 }, { 4, 5, 6 } };
   int array2[ 2 ][ 3 ] = { 1, 2, 3, 4, 5 };
   int array3[ 2 ][ 3 ] = { { 1, 2 }, { 4 } };

   cout << "Values in array1 by row are:" << endl;
   printArray( array1 );

   cout << "\nValues in array2 by row are:" << endl;
   printArray( array2 );

   cout << "\nValues in array3 by row are:" << endl;
   printArray( array3 );
   return 0; // indicates successful termination
} // end main

// output array with two rows and three columns
void printArray( const int a[][ 3 ] )
{
   // loop through array's rows
   for ( int i = 0; i < 2; i++ )
   {
      // loop through columns of current row
      for ( int j = 0; j < 3; j++ )
         cout << a[ i ][ j ] << ' ';

      cout << endl; // start new line of output
   } // end outer for
} // end function printArray
```

```
Values in array1 by row are:
1 2 3
4 5 6

Values in array2 by row are:
1 2 3
4 5 0

Values in array3 by row are:
1 2 0
4 0 0
```

Common M-D Array Manipulation


```
for ( int col = 0; col < 4; col++ )
     a[ 2 ][ col ] = 0;
```

- What the previous code does?

```
total = 0;
for ( row = 0; row < 3; row++ )
    for ( col = 0; col < 4; col++ )
       total += a[ row ][ col ];
```

- What the previous code does?

End, Q & A

Computer Programming (2)

Lecture No.2: **Functions**

Lecturer: Dr. Khaled Mohammed bin Abdl
Department of Information Technology,
College of Applied Science,
Seyion University

Semester 2, 2023

Content

- C++ Modules
- Function syntax
- Function parameters and return types
- Defining functions within programs
- Using and invoking function
- Common errors

Modules

- Blocks of codes with an independent functionality
- Written and tested separately
- Combined to form a complete program
- Modules either control or general
  - Control
    - Called “Main”
      - each program include only one control module
  - General
    - Perform manipulations on data e.g. calculation, print, read and validation

C++ Modules

- All Modules in C++ are called “functions”
  - called methods or procedures in other languages
- Functions can be
  - Programmer-defined
  - Built-in: pow, sqrt, .. etc
- Advantage
  - make programs easier to write, test, debug and maintain

Functions Syntax

Pseudo Code

```
ModuleName(Parameter List)

1.
2.
3.
4.   Exit/Return varaible
```

```
ReturnType FctName(Parameter List)
{
  action1;
  action2;
   ..
  actionN;
  return value   ;
}
```

1 2 3

Function Header

1

2

Function Body

Function Name and Return Type

- Function name follows rules of naming identifiers as variables and constants
- Function return ONLY one value
- Function return types
  - Built-in data types
  - User-defined data types
  - Nothing is returned by using keyword `void`

```
int f1(Parameter List)
{

   return value   ;

}
```

```
void f2(Parameter List)
{

   return;

}
```

```
void f2(Parameter List)
{



}
```

Function Parameters

- Provide the communication links between the main program and its functions
- Parameters declared in the function header within parenthesis
- Either have a single, multiple parameters or have no parameters
- List of a parameters are separated by commas
- Each parameter must be defined as follows

```
DataType   parameterName
```

```
char f1(int x, int y)
{

   return value  ;

}
```

Problem

Write a program that asks the user for the exam grade and prints a message indicating if the user has passed the exam or not.

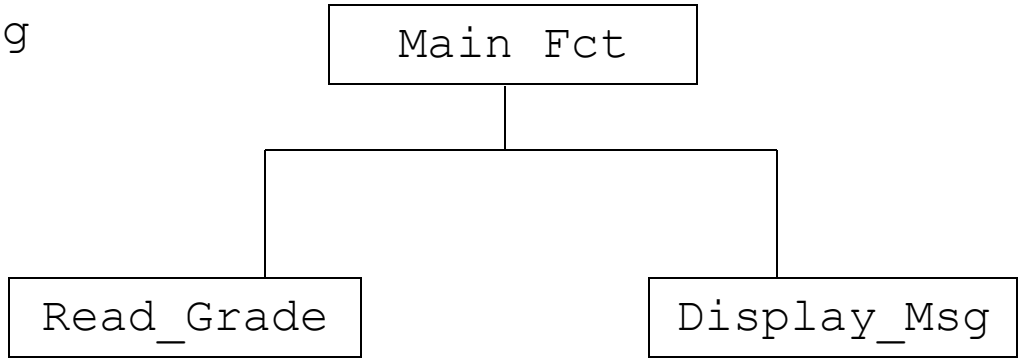
```
#include <iostream>
using namespace std;

int main()
{
        int grade;

        cout<<"Enter your grade:\n";
        cin>>grade;

        if (grade>60)  cout<<"You passed";
        else   cout<<"You failed";

        return 0;
}
```

Can you solve the problem using functions ?

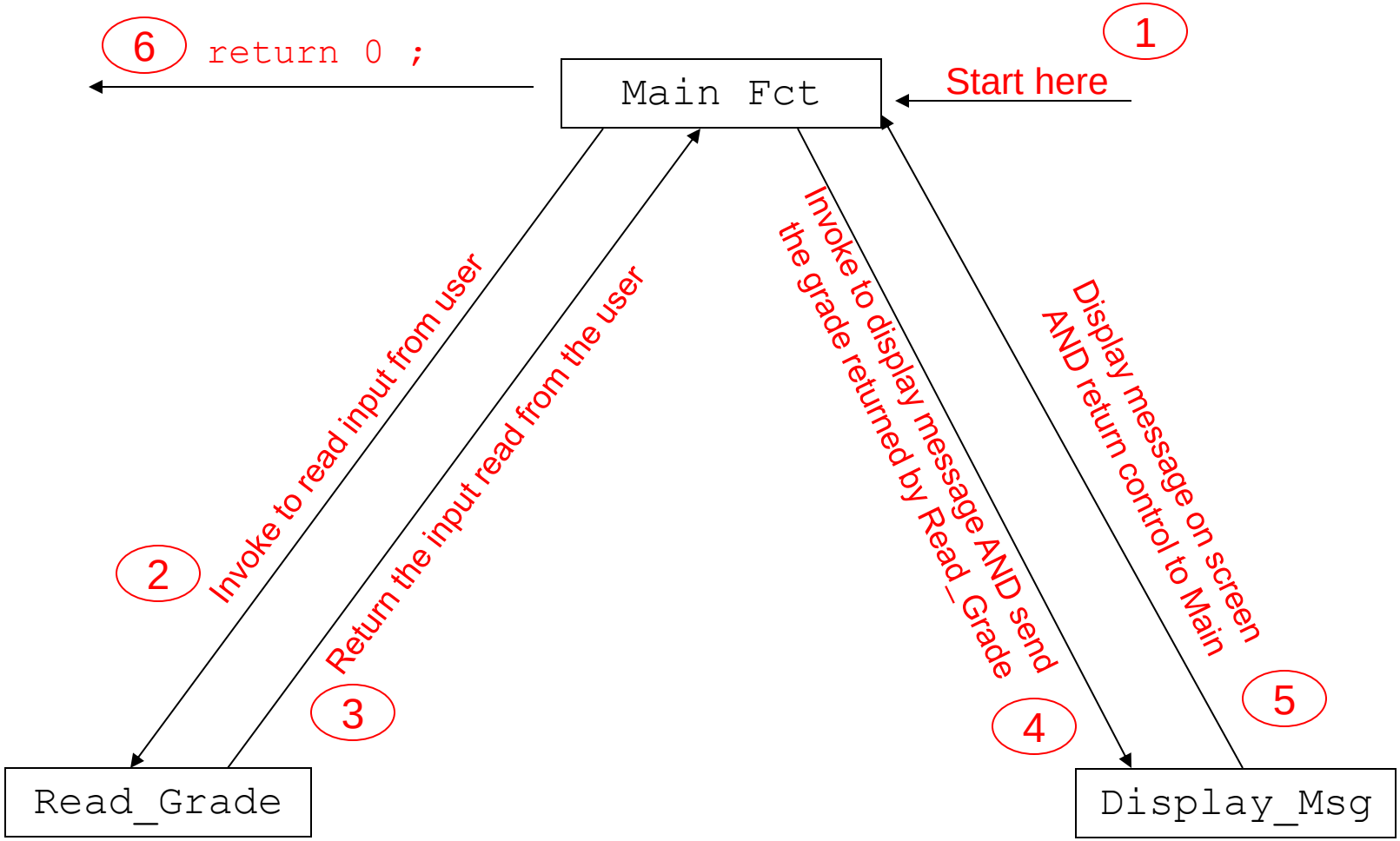
1- Selecting Functions

- Divide the program into tasks
  - Read student grade
  - Print appropriate message

- Naming functions
  - `Read_Grade`
  - `Display_Msg`
  - `main`



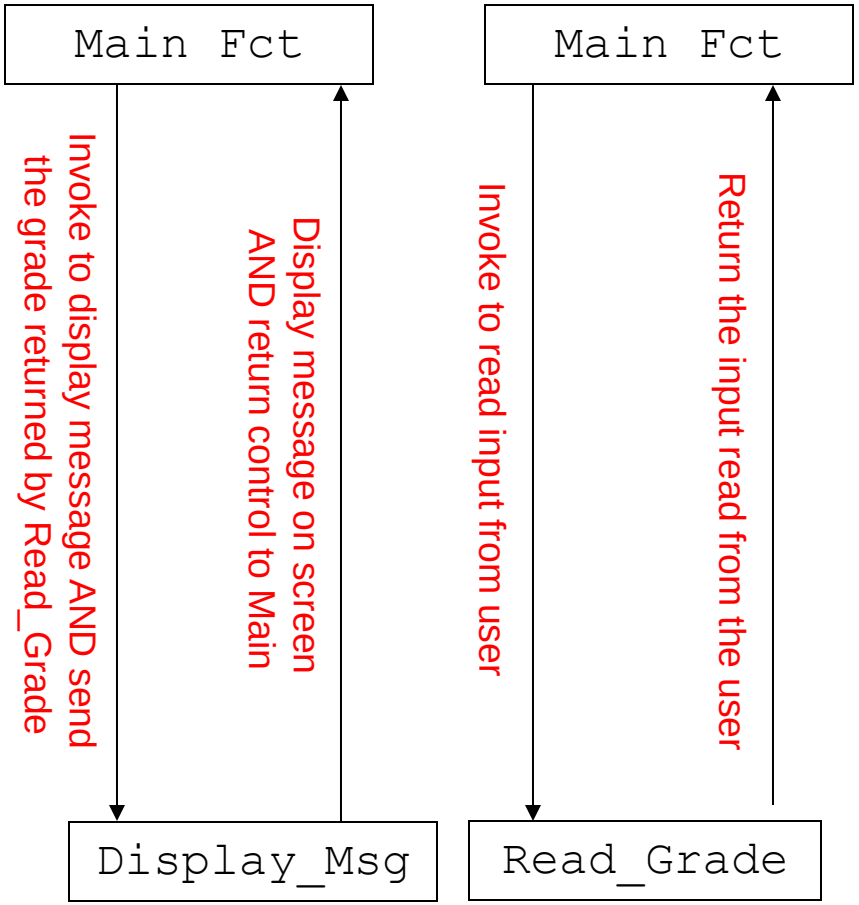
2-Analyzing Flow of Control



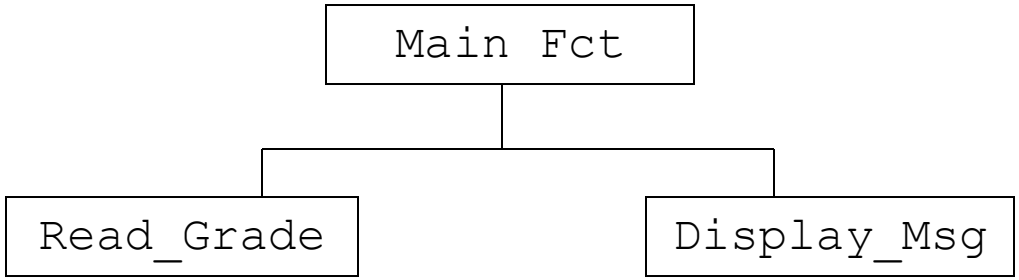
3-Defining Functions

- For each function decide on
  - Possible Parameter List
  - Possible Return Value

```
int   Read_Grade(){

}


void   Display_Message(int g){

}
```



3-Defining Functions (cont.)




```
#include <iostream>
using namespace std;

int main()
{
      int grade;                                  Read_Grade

      cout<<"Enter your grade:\n";
      cin>>grade;

      if (grade>60)  cout<<"You passed";
      else   cout<<"You failed";
                                                  Display_Msg
      return 0;
}
```

3-Defining Functions (cont.)


```
int Read_Grade()
{
        int grade;
        //grade is a local variable
        cout<<"Enter your grade:\n";
        cin>>grade;

        return grade;
}
```

grade scope

```
void Display_Msg(int g)
{
        if (g>60)   cout<<"You passed";
        else    cout<<"You failed";
}
```

g scope

4-Using Functions


```
#include <iostream>
using namespace std;
// ====== You Must Define your Modules before Main ======
int Read_Grade()
{
      int grade;
      //grade is a local variable
      cout<<"Enter your grade:\n";
      cin>>grade;

      return grade;
}
void Display_Msg(int g)
{
      if (g>60)  cout<<"You passed";
      else   cout<<"You failed";
}
// ====================== Main Fct =====================
int main()
{
      // Invoke read module and store returned input
      // Invoke display module and pass the grade
      return 0;
}
```

4-Using Functions (cont.)

- In our example, all invokations occur in Main function
- Invoking C++ functions syntax
  - Write function name followed by ()
  - Within ( ) passing values (arguments) to parameters
    - Functions could have
      - No parameters → **`FunctionName()`**
      - One or more parameters → **`FunctionName(v1,v2)`**
  - Capturing returned values
    - Functions return type could be
      - **`void`** → **`FunctionName() OR FunctionName(v1,v2)`**
      - **`Non-void`** → must use returned value in expression

4-Using Functions (cont.)

- Capturing returned values
  - **`Non-void`** ➔ must use returned value in expression

- Function **`Read_grade()`** returns an **`int`**
  - `int n = Read_Grade();`
  - `if ( Read_Grade() == 3 )` …… … …
  - `cout << Read_Grade();`

4-Using Functions (cont.)

- Each value passed in a function call is referred to as an Argument
- Arguments (Passed in a Function call) and parameters (defined in a function header) **MUST** match in
  - Number
  - Order
  - Data types

Parameter

```
void Display_Msg(int g)
{
    if (g > 60)
       cout<<"You passed";
    else
       cout<<"You failed";
}
```

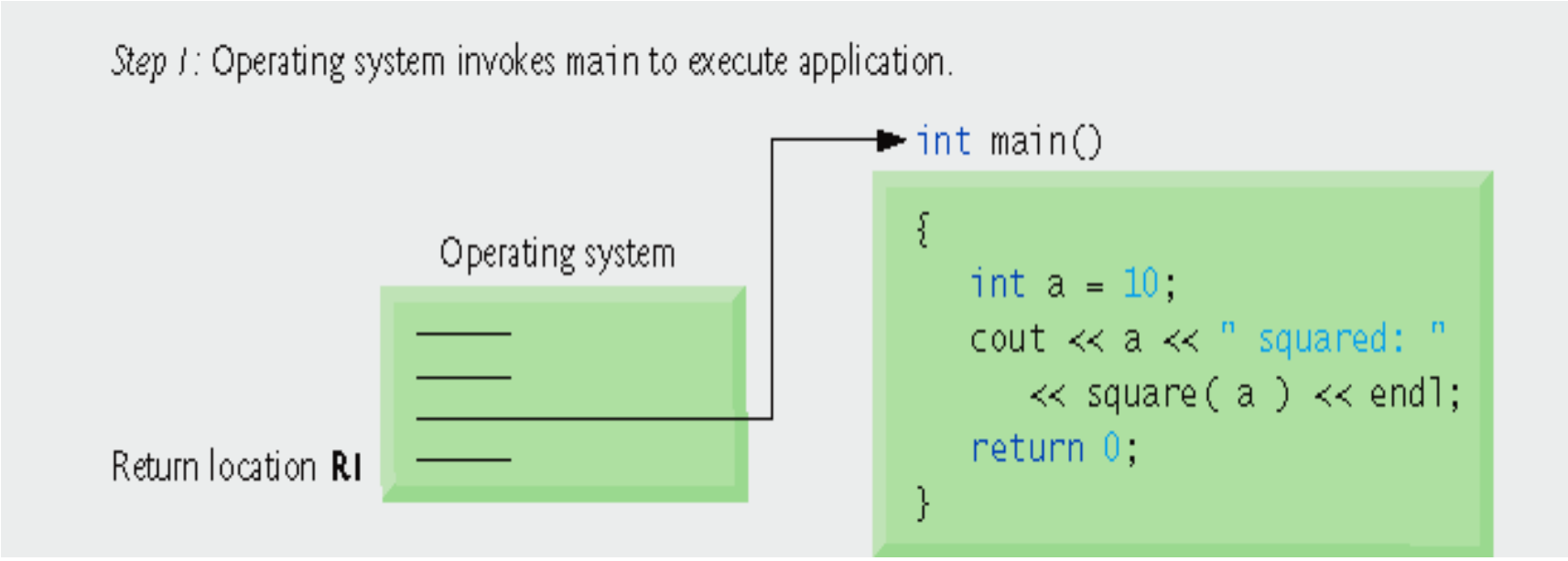
Argument

```
int main()
{
        Display_Msg(87);

        return 0;
}
```

Final Solution


```
#include <iostream>
using namespace std;
// ====== You Must Define your Modules before Main ======
int Read_Grade()
{
      int grade;
      //grade is a local variable
      cout<<"Enter your grade:\n";
      cin>>grade;

      return grade;
}
void Display_Msg(int g)
{
      if (g>60)  cout<<"You passed";
      else   cout<<"You failed";
}
// ====================== Main Fct =====================
int main()
{
      int n = Read_Grade(); // Invoke read function
      Display_Msg(n);       // Invoke display function
      return 0;
}
```

How Function Call Works (1)



How Function Call Works (2)

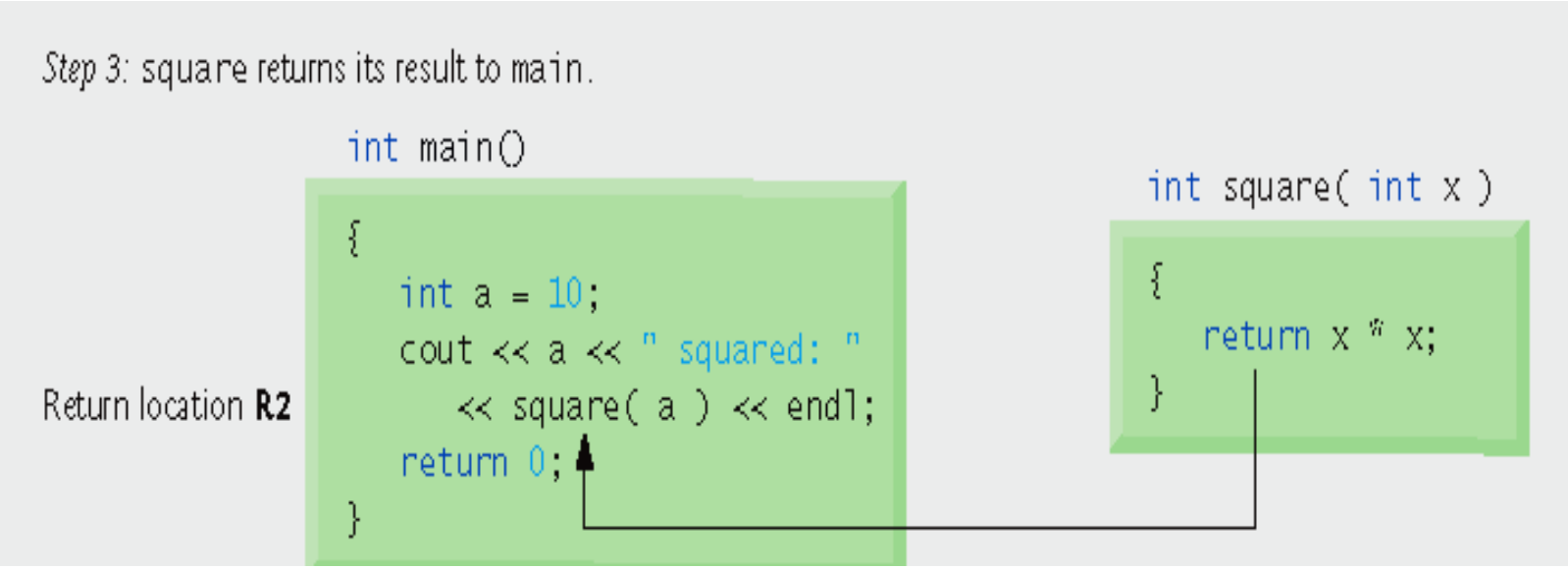
Step 2: main invokes function square to perform calculation.

```
int main()
{
   int a = 10;
   cout << a << " squared: "
      << square( a ) << endl;
   return 0;
}
```

Return location **R2**

```
int square( int x )
{
   return x * x;
}
```

How Function Call Works (3)



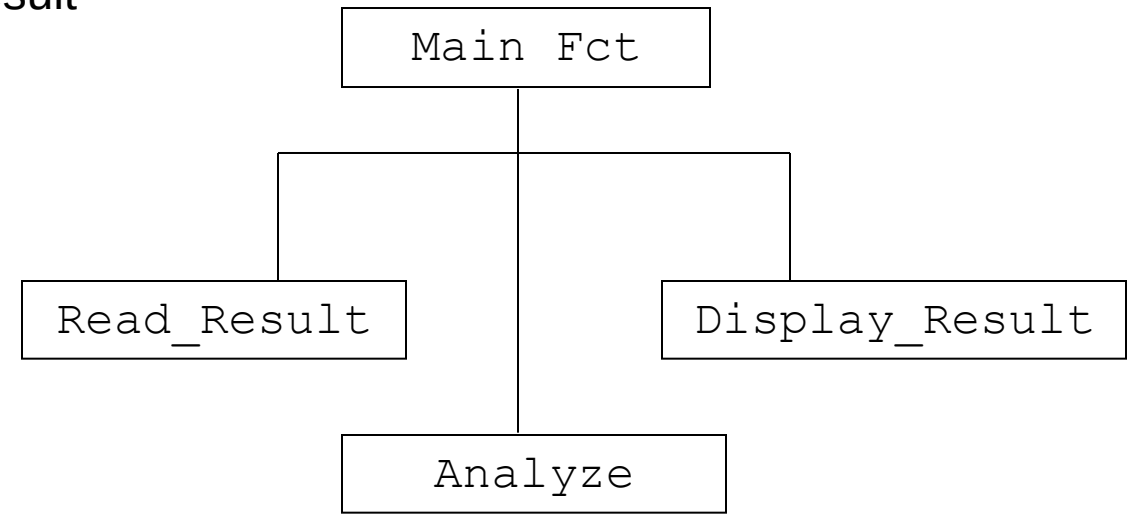
Problem # 2

A college has a list of test results (1 = pass, 2 = fail) for 10 students. Write a program that analyzes the results. If more than 8 students pass, print "Raise Tuition“.

We want to solve this problem using Modules (Functions)

Selecting Modules

- Divide the program into tasks
  - Reading student result
  - Analyzing student result
  - Displaying results
- Naming functions
  - `Read_Result`
  - `Analyze`
  - `Display_Result`
  - `Main`



```
// Fig. 2.11: fig02_11.cpp
// Analysis of examination results.
#include <iostream>

using std::cout;
using std::cin;
using std::endl;

// function main begins program execution
int main()
{
   // initialize variables in declarations
   int passes = 0;            // number of passes
   int failures = 0;          // number of failures
   int studentCounter = 1;    // student counter
   int result;                // one exam result

   // process 10 students using counter-controlled loop
   while ( studentCounter <= 10 ) {
```

Read_Result

```
      // prompt user for input and obtain value from user
      cout << "Enter result (1 = pass, 2 = fail): ";
      cin >> result;

```

```
25      // if result 1, increment passes; if/else nested in while
26      if ( result == 1 )         // if/else nested in while
27         passes = passes + 1;
28
29      else  // if result not 1, increment failures
30         failures = failures + 1;
31                                                          Analyze
32      // increment studentCounter so loop eventually terminates
33      studentCounter = studentCounter + 1;
34
35   } // end while
36
37   // termination phase; display number of passes and failures
38   cout << "Passed " << passes << endl;
39   cout << "Failed " << failures << endl;
40
41   // if more than eight students passed, print "raise tuition"
42   if ( passes > 8 )
43      cout << "Raise tuition " << endl;
44                                                          Display_Result
45   return 0;   // successful termination
46
47 } // end function main
```

Using Functions


```
#include <iostream>
using namespace std;

// ======================= Main Fct =======================
int main()
{
      int passes=0, failures=0, student_counter=1;
      int result;

      // process 10 students
      while (student_counter <= 10)
      {
          // invoke Read_Result function
          // invoke Analyze function

          student_counter++;
      }

      // invoke Display_Result function

      return 0;
}
```

```
int  Read_Result(){

    int res;
    cout<<"Enter Result ( 1=Pass, 2=Fail): ";
    cin>> res;
    return res;

}

int main()
 {
     int passes=0, failures=0, student_counter=1;
     int result;

     // process 10 students
     while (student_counter <= 10)
     {
          result = Read_Result(); // invoke Read_Result function
          // invoke Analyze function

          student_counter++;
     }

     // invoke Display_Result function

     return 0;
 }
```

Take out to the Global Scope

```
?                    int result
---------  Analyze(---------------){

                          ?
    if (result==1)  passes++;
    else failures++;
            ?
}
```

scope

ONLY one value can return !!!!!!

```
int main()
 {
    int passes=0, failures=0, student_counter=1;
    int result;

    // process 10 students
    while (student_counter <= 10)
    {
         result = Read_Result(); // invoke Read_Result function
         Analyze(result);        // invoke Analyze function

         student_counter++;
    }

    // invoke Display_Result function

    return 0;
 }
```

scope

```
int passes=0, failures=0;

void          int result
---------  Analyze(--------------){

     if (result==1)  passes++;
     else failures++;

}
```

scope

```
int main()
 {
----------- int student_counter=1;
     int result;

     // process 10 students
     while (student_counter <= 10)
     {
          result = Read_Result(); // invoke Read_Result function
          Analyze(result);        // invoke Analyze function

          student_counter++;
     }

     // invoke Display_Result function

     return 0;
 }
```

scope

```
int passes=0, failures=0;

 void
---------  Display_Result(---------------){

    cout<<"Passed " << passes << endl;
    cout<<"Failures " << failures << endl;

    if ( passes > 8 )
        cout<<"Raise Tution" << endl;
}

int main()
 {
    int student_counter=1;
    int result;

    // process 10 students
    while (student_counter <= 10)
    {
        result = Read_Result(); // invoke Read_Result function
        Analyze(result);        // invoke Analyze function

        student_counter++;
    }

    Display_Result();// invoke Display_Result function

    return 0;
 }
```

Make it empty

Using Library Functions

- Need to know the points below to write invokation of built-in functions correctly
  - Name of the function
  - Number , type and order of parameters
  - Type of return value if it has
  - Header file

- Examples

Header File	Function call	Example	Value
`<cmath>`	`pow(x,y)`	`pow(2,3)`	`8`
`<iomanip>`	`setprecision(n)`	`setprecision(2)<< 1.2345`	`1.23`

Useful Standard Library Function

- Standard library function **`std::pow( )`**
  - Calculates an exponent
  - Function takes two arguments of type **`double`** and returns a **`double`** value.
  - Requires header file `<cmath>`
  - Example :

```
#include <cmath>
using std::pow;

//in main function
double x,y,result;
x=2;   y=2;
result = pow(x,y);
```

Calculates the value of x raised to the yth power

Function	Description	Example
`ceil( x )`	rounds x to the smallest integer not less than x	`ceil( 9.2 )` is `10.0` `ceil( -9.8 )` is `-9.0`
`cos( x )`	trigonometric cosine of x (x in radians)	`cos( 0.0 )` is `1.0`
`exp( x )`	exponential function e^x	`exp( 1.0 )` is `2.71828` `exp( 2.0 )` is `7.38906`

`fabs( x )`	absolute value of x	`fabs( 5.1 )` is `5.1`
		`fabs( 0.0 )` is `0.0`
		`fabs( -8.76 )` is `8.76`

`floor( x )`	rounds x to the largest integer not greater than x	`floor( 9.2 )` is `9.0` `floor( -9.8 )` is `-10.0`
`fmod( x, y )`	remainder of x/y as a floating-point number	`fmod( 2.6, 1.2 )` is `0.2`
`log( x )`	natural logarithm of x (base e)	`log( 2.718282 )` is `1.0` `log( 7.389056 )` is `2.0`
`log10( x )`	logarithm of x (base 10)	`log10( 10.0 )` is `1.0` `log10( 100.0 )` is `2.0`
`pow( x, y )`	x raised to power y (x^y)	`pow( 2, 7 )` is `128` `pow( 9, .5 )` is `3`
`sin( x )`	trigonometric sine of x (x in radians)	`sin( 0.0 )` is `0`
`sqrt( x )`	square root of x (where x is a nonnegative value)	`sqrt( 9.0 )` is `3.0`
`tan( x )`	trigonometric tangent of x (x in radians)	`tan( 0.0 )` is `0`

Math library functions

Common Errors

- Compilation errors
  - Declaring function has parameters of the same type as char a,b instead of char a, char b
  - Mismatching function header with function call in the number, type and order of parameters and arguments
  - Invoking function before it is defined, e.g. when f1 invokes f2, f2 must be defined before f1
  - Attempting to use a function that has no return value in an expression
- Logic errors
  - Undesired implicit typecasting of arguments when they assigned to parameters

Exercise - 1

- Write a function that test a received 2 integer numbers then return the sum then print the result

End, Q & A